ORGANIC DRIVEN DEVELOPMENT FRENTE A SPEC-DRIVEN DEVELOPMENT EN EL DESARROLLO ASISTIDO POR AGENTES: ESTUDIO DE CASO DE UN SIMULADOR CONCURRENTE EN PYTHON

Integrantes:

Santiago ALVAREZ [1]	salvarez927@alumno.unlam.edu.ar
Victoria BLANCO [2]	viblanco@alumno.unlam.edu.ar
Santiago BRIASCO CALVO [3]	sbriascocalvo@alumno.unlam.edu.ar
Octavio Dion GEMELLARO [4]	ogemellaro@alumno.unlam.edu.ar
Micaela Abril MARRONE [5]	mmarrone584@alumno.unlam.edu.ar
Tomas Agustín MENENDEZ [6]	tmenendez@alumno.unlam.edu.ar

Asignatura:

(01-3900) PROGRAMACIÓN CONCURRENTE

Universidad Nacional de La Matanza — Departamento de Ingeniería e Investigaciones Tecnológicas

Fecha - Versión:

19/10/2026 – Versión 2.00

Actividad Práctica Obligatoria N.° 1 — 2.° cuatrimestre 2026

Resumen:

Este trabajo compara dos flujos de trabajo para el desarrollo de software asistido por agentes de IA: Spec-Driven Development (SDD), basado en especificaciones formales escritas antes del código, y Organic Driven Development (ODD), el protocolo con el que la herramienta gentle-ai reemplazó a SDD en su versión 4.0.0. Se plantea la hipótesis de que ODD conserva los mecanismos de aseguramiento de SDD —criterios de aceptación explícitos, verificación test-first, revisión independiente y trazabilidad requisito → tarea → prueba → commit— y genera artefactos solo para el trabajo sustancial, mientras delega la continuidad entre sesiones en la memoria persistente Engram. El diseño combina una comparación conceptual con criterios explícitos y un estudio de caso exploratorio: cinco fases del desarrollo incremental de un simulador concurrente en Python (1154 líneas de producción, 1292 de pruebas, 83 pruebas). No hay grupo de control: el equipo no usó SDD porque ya había sido retirado. La hipótesis resulta parcialmente sustentada. En las dos fases con documento ODD se observó la cadena completa de trazabilidad y ciclos RED → GREEN registrados; en la última, además, las revisiones independientes detectaron cinco defectos que las pruebas no cubrían. En las otras tres fases no hubo documento, porque el cumplimiento de ODD depende de instrucciones en el prompt. Engram aportó continuidad verificable entre sesiones, con costos de fricción en la revisión y en la configuración del entorno.

Palabras Clave: *desarrollo asistido por agentes, spec-driven development, organic driven development, memoria persistente, revisión de código, programación concurrente*

I. CONTEXTO

El trabajo se inscribe en el área de Ingeniería de Software, en la intersección entre la programación concurrente, la generación de código con modelos de lenguaje de gran escala (LLM) y los procesos de desarrollo asistidos por agentes. Corresponde a la Actividad Práctica Obligatoria N.° 1 de la asignatura Programación Concurrente de la carrera de Ingeniería en Informática de la Universidad Nacional de La Matanza (UNLaM), dictada en el segundo cuatrimestre de 2026, y amplía la versión 1.00 del informe del grupo, centrada en las métricas del programa generado [1]. El proyecto no cuenta con financiamiento externo.

El objeto de estudio ya no es el programa en sí, sino el proceso con el que se construyó. El código se desarrolló con Claude Code y el modelo Claude Opus 5.5 (Anthropic), configurados con gentle-ai 4.0.0 [2] y con la memoria persistente Engram 3.3.0 [3]. Al cierre de este informe, las ramas principales de esas herramientas ya están en gentle-ai v4.2.0 y Engram v3.4.0. La versión 4.2.0 modificó las reglas de clasificación y delegación de ODD [4], por lo que todo lo que aquí se describe corresponde al comportamiento de la versión 4.0.0. El código, las pruebas y el historial están en el repositorio del grupo, `https://github.com/UNLAM-PROG-C/2026-PROGC-Q2-M4/tree/develop/TP%20Metricas`.

II. INTRODUCCIÓN

A. Concurrencia y generación de código con LLM

La programación concurrente organiza un programa como un conjunto de actividades que progresan de manera intercalada y comparten recursos. Sus fundamentos son los procesos secuenciales cooperantes y los semáforos de Dijkstra [5] y los monitores de Hoare [6]. Sus errores son difíciles de detectar: en 105 errores reales de aplicaciones de código abierto, la mayoría de los que no eran interbloqueos resultaron violaciones de atomicidad o de orden que dependían de intercalados poco frecuentes [7]. Por eso se privilegian abstracciones de alto nivel —colas bloqueantes, objetos inmutables, señales de terminación— frente al manejo directo de candados [8].

Los LLM se convirtieron en asistentes de programación de uso masivo, y la generación de código es la tarea de ingeniería de software más estudiada con ellos [9]. Sin embargo, su salida no es confiable por sí misma: cerca del 40 % de los programas generados por GitHub Copilot en escenarios de riesgo contenía vulnerabilidades [10], y su desempeño al generar código paralelo es claramente inferior al que logran en código secuencial [11]. La primera versión de este informe llegó a una conclusión compatible: la lógica central de sincronización fue correcta, pero los defectos aparecieron en los bordes —portabilidad, interrupciones durante el cierre, requisitos contradictorios— y se detectaron con pruebas y revisiones, no por la sola generación [1]. Esa observación motiva la pregunta de este trabajo: qué proceso de desarrollo conviene envolver alrededor del agente.

B. Del vibe coding al desarrollo agéntico

El término *vibe coding*, popularizado en 2025 [12], describe un estilo en el que el desarrollador guía al modelo en lenguaje natural y evalúa el resultado por su comportamiento más que por la lectura del código. Sapkota et al. [13] lo

distinguen del *agentic coding*, en el que el modelo actúa como un agente que planifica, ejecuta herramientas y verifica su propio trabajo con supervisión humana acotada. La base de ese paradigma es el ciclo ReAct, que intercala razonamiento y acciones sobre el entorno [14]. SWE-agent mostró que el diseño de la interfaz entre el agente y la computadora —qué comandos ve, cómo recibe la retroalimentación— influye en el desempeño tanto como el modelo [15], y SWE-bench estableció un banco de pruebas basado en *issues* reales de GitHub para medirlo [16].

Esos resultados trasladan la atención del modelo al *arnés* que lo rodea: las instrucciones de sistema, las herramientas, la memoria y el flujo de trabajo. Las metodologías que se comparan en este informe pertenecen a esa capa. No cambian el modelo, sino qué artefactos produce el agente, en qué orden y con qué controles.

C. Spec-Driven Development

En la industria, Spec-Driven Development (SDD) designa un enfoque en el que se escribe una especificación antes de que la IA escriba el código, y esa especificación funciona como fuente de verdad para el humano y para el agente. Böckeler [17] distingue tres niveles: *spec-first* (la especificación guía una tarea y luego puede descartarse), *spec-anchored* (se mantiene y evoluciona junto con el código) y *spec-as-source* (la especificación es el artefacto principal y el código se deriva de ella). GitHub Spec Kit organiza el trabajo en las etapas Specify → Plan → Tasks → Implement, guiadas por los comandos `/specify`, `/plan` y `/tasks` [18]. Kiro, por su parte, genera para cada especificación los archivos `requirements.md`, `design.md` y `tasks.md` [19]. No se encontró literatura revisada por pares específica sobre SDD con LLM, por lo que el marco se apoya en estas fuentes de la industria.

En gentle-ai, y en su predecesora Agent Teams Lite [20], SDD era un flujo de nueve fases: init, explore, propose, spec, design, tasks, apply, verify y archive, con una fase opcional de investigación agregada luego. Las fases formaban un grafo de dependencias en el que la especificación y el diseño podían ejecutarse en paralelo. Cada cambio dejaba en `openspec/changes/<cambio>/` una propuesta (por qué, alcance y enfoque), especificaciones delta con palabras clave RFC 2119 y escenarios Given/When/Then, un diseño y una lista de tareas. La fase archive fusionaba los deltas en `openspec/specs/`, que describía «cómo funciona el sistema hoy» [20]. En la clasificación de Böckeler, esa base viva de especificaciones lo ubica en el nivel *spec-anchored*. La topología era la de un orquestador que solo delega y lanza un sub-agente con contexto nuevo por fase, pide aprobación entre fases y recibe de cada uno un sobre de resultado con estado, resumen, artefactos y riesgos [20].

El retiro de SDD fue gradual. En la versión 3.0.0 (16 sep. 2026) ODD pasó a ser el protocolo por defecto, y SDD quedó como alternativa que solo se elegía de forma explícita, porque su ciclo de vida «se rehízo para quitar ceremonia que se había vuelto control de acceso en lugar de valor» [21]. El issue #4612 explica el motivo: SDD se había convertido en «un segundo sistema de gobierno de la ejecución», con admisión y cierre obligatorios de intentos e informes de verificación previos al archivo, que creaban «lugares evitables donde el trabajo útil puede quedar bloqueado por el estado del flujo y no por un problema real de producto, autorización o técnica» [22]. El issue #4959 descartó mantenerlo como opción [23], y la versión 4.0.0 (1 oct. 2026), titulada «SDD Retires, ODD Leads», eliminó los comandos `/sdd-*`, los perfiles y la integración con OpenSpec. Según sus notas, la mayor parte de los 1576 archivos modificados en esa versión corresponden a ese retiro [24].

D. Organic Driven Development

ODD parte de un diagnóstico opuesto al de SDD: «los cambios pequeños no deberían necesitar una línea de planificación, y el trabajo más grande no debería perder su contexto entre sesiones» [2]. Su documentación lo define como una guía compartida para el agente y «no un nuevo CLI, motor de estado ni fase de planificación obligatoria» [25]. Técnicamente es texto que gentle-ai inyecta en las instrucciones de sistema de cada agente. El issue que lo volvió obligatorio excluyó de manera expresa la aplicación en tiempo de ejecución mediante hooks o arneses [26], y el que lo introdujo advertía que «convertirlo en otro arnés obligatorio resolvería el problema equivocado» [27].

En la versión 4.0.0, el protocolo se aplica en siete pasos a cada pedido [25], [26]:

- **Autorizar:** decidir si el pedido autoriza un cambio; las explicaciones, investigaciones y propuestas son de solo lectura.
- **Explorar:** leer el código y los requisitos existentes, en proporción al pedido.
- **Resolver la incertidumbre:** investigación opcional solo para una incertidumbre nombrada, una única pregunta para una decisión de producto real y, como máximo, un desafío independiente a una premisa de alto impacto.
- **Clasificar:** en la versión 4.0.0, el trabajo es sustancial cuando tiene dos o más pasos de implementación significativos o un progreso que valga la pena recuperar tras una interrupción.
- **Registrar antes de la primera escritura:** para el trabajo sustancial, crear `odd/tasks/<funcionalidad>.md` y su espejo en Engram con la clave `odd/<funcionalidad>/tasks`.
- **Implementar tarea por tarea:** con la topología mínima, política test-first, verificaciones con evidencia observada y un commit por unidad de trabajo.
- **Cerrar:** informar el resultado verificado, cada verificación fallida, omitida o pendiente y el siguiente paso.

El documento de funcionalidad reúne objetivo, alcance, restricciones, una lista de tareas con identificadores estables, criterios de aceptación, la ruta elegida para cada tarea y la evidencia. El trabajo pequeño y entendido no genera artefactos durables. La política test-first exige observar la prueba en rojo (RED) antes de implementar, llevarla a verde (GREEN) y refactorizar, siempre que exista una prueba determinista ejecutable y un resultado esperado claro [25]; desde la versión 4.0.0 ya no es una opción separada, sino la conducta por defecto [24]. Cada tarea cierra con un commit en formato Conventional Commits sobre una rama de funcionalidad [28]. El umbral de unas 400 líneas modificadas por tarea es solo una heurística de planificación, no un límite ni un disparador de revisión, y por encima de ese volumen se aplica una estrategia de entrega (`ask-on-risk` por defecto, `auto-chain`, `single-pr` o `exception-ok`) que puede dividir el trabajo en pull requests encadenados [25].

La delegación también cambió de criterio. En la versión 4.0.0 el trabajo permanece en el agente principal mientras la evidencia necesaria entre en un lote paralelo de hasta tres llamadas y unos 10 000 tokens; si no, un explorador de solo lectura devuelve un resumen acotado con referencias `ruta:línea` [24]. El prompt instalado agrega disparadores obligatorios: un escritor delegado cuando se modifican dos o más archivos no triviales y un límite orientativo de unos 150 000 tokens de contexto por sesión. En SDD, en cambio, la delegación era la regla.

E. Receipt-Driven Development y revisión nativa

La revisión independiente que SDD ubicaba en la fase verify pasó en gentle-ai a un mecanismo separado, Receipt-Driven Development (RDD), que «revisa un candidato terminado sin asumir la propiedad de la entrega»: el binario congela un candidato del árbol de trabajo, coordina una revisión acotada, «quema» la autorización obtenida y devuelve el control al humano [29]. El usuario lo activa o desactiva con `gentle-ai review mode`, y está activo por defecto desde la versión 3.5.0 [30]. Tras cada commit de unidad de trabajo, el comando `review assess` clasifica el riesgo del cambio según lo que modificó y no según su tamaño. Un cambio pasivo solo recibe una relectura estructural. Uno de riesgo medio recibe una lente de revisión (por defecto, fiabilidad) y requiere el consentimiento del usuario. Uno de riesgo alto recibe las cuatro lentes canónicas —riesgo, resiliencia, legibilidad y fiabilidad—, consentimiento y un pronóstico de costo [29]. Los hallazgos graves de carácter inferencial pasan por un único lote de refutación de solo lectura, y solo los causados por el candidato pueden bloquear. Se admite una única corrección acotada, validada sobre los árboles congelados. La aprobación es informativa: nunca hace commit, push ni pull request [29].

F. Memoria persistente para agentes

Un agente basado en LLM pierde todo lo que no está en su ventana de contexto, y aun dentro de ella su uso de la información no es uniforme: Liu et al. mostraron que el desempeño cae cuando el dato relevante está en el medio de un contexto largo [31]. Dos líneas de trabajo respondieron con memoria externa. MemGPT trata al LLM como un sistema operativo con una jerarquía de memoria y pagina información entre el contexto y un almacenamiento externo [32]. Los Generative Agents registran un flujo de memoria del que recuperan y sobre el que reflexionan para sostener un comportamiento coherente en el tiempo [33].

Engram aplica esa idea a los agentes de programación [3]. Es un único binario en Go que expone herramientas MCP sobre una base SQLite con búsqueda de texto completo FTS5. Su unidad es la *observación*, curada por el propio agente con la estructura qué / por qué / dónde / aprendido y un tipo (decisión, corrección de error, descubrimiento, etc.); su filosofía es que el agente decide qué vale la pena recordar, en lugar de capturar todas las llamadas a herramientas. Una *clave de tema* (`topic_key`) convierte el guardado en una actualización del mismo registro e incrementa su contador de revisiones. Las *sesiones* cierran con un resumen estructurado (objetivo, descubrimientos, logros, próximos pasos, archivos) que la sesión siguiente recupera. Cuando una observación nueva se parece a otra existente, Engram pide al agente que juzgue la relación (`related`, `compatible`, `scoped`, `conflicts_with`, `supersedes` o `not_conflict`) y la registra; por debajo de una confianza de 0,7 el agente debería consultar al usuario. La captura pasiva guarda como observaciones separadas los ítems de las secciones «Key Learnings» que devuelven los sub-agentes. Tras una compactación del contexto, el protocolo obliga a guardar primero el resumen compactado y luego recuperar el contexto. La sincronización es local primero, con exportación en fragmentos versionables en Git o replicación opcional en la nube [3].

G. Problema, preguntas de investigación e hipótesis

El retiro de SDD en gentle-ai plantea una pregunta que la documentación de la herramienta responde solo desde la intención de sus autores: si un flujo liviano, que genera artefactos únicamente cuando hacen falta, conserva las garantías que el flujo basado en especificaciones obtenía con su ceremonia. Este trabajo aborda esa pregunta con tres preguntas de investigación:

- **PI1.** ¿En qué difieren conceptualmente ODD y SDD en cuanto a unidad de planificación, artefactos, verificación, revisión, persistencia y topología?
- **PI2.** ¿Proporcionó ODD, en el caso estudiado, los mecanismos de aseguramiento que SDD obtenía con sus especificaciones?
- **PI3.** ¿Qué costos y fricciones introdujo ODD, junto con RDD y Engram, en ese caso?

Hipótesis (H). En un desarrollo incremental de software concurrente asistido por agentes, ODD conserva los mecanismos de aseguramiento que SDD obtenía mediante especificaciones formales —criterios de aceptación explícitos, verificación test-first, revisión independiente y trazabilidad requisito → tarea → prueba → commit—, genera artefactos solo cuando la tarea es sustancial y delega la continuidad entre sesiones en la memoria persistente (Engram).

III. MÉTODOS

A. Diseño de la investigación

Se adoptó un diseño mixto de carácter exploratorio. La PI1 se responde con una comparación conceptual basada en la documentación primaria de ambos flujos (repositorios, notas de versión e *issues* de gentle-ai y de Agent Teams Lite) y en criterios definidos antes del análisis. Las PI2 y PI3 se responden con un estudio de caso único sobre el desarrollo del simulador. La comparación entre ODD y SDD no es empírica: el equipo nunca usó SDD, porque gentle-ai 4.0.0, publicado el 1 de octubre de 2026, ya lo había retirado cuando comenzó el desarrollo (9 de octubre). El caso, por lo tanto, no contrasta dos tratamientos; evalúa si los mecanismos que SDD declaraba aparecen de hecho en un proceso guiado por ODD.

B. Unidad de análisis y entorno

La unidad de análisis es el desarrollo del simulador de centro logístico del grupo: camiones productores que descargan paquetes en un muelle `queue.Queue` acotado, operarios consumidores que los registran en un inventario protegido por un candado, lectores (panel de terminal y exportador JSON) que trabajan sobre instantáneas inmutables, un supervisor que impone pausas de mantenimiento aleatorias y un cierre ordenado en fases [1]. El estado final tiene 1154 líneas de producción en 15 archivos Python y 1292 líneas de pruebas en 13 archivos, con 83 pruebas `unittest`. El desarrollo se hizo en cinco fases, cada una en una rama propia integrada en `DesarrolloAPO` con merge sin avance rápido (Tabla I). Todas las fases usaron el mismo agente (Claude Code), el mismo modelo (Claude Opus 5.5) y las mismas versiones de gentle-ai (4.0.0) y Engram (3.3.0), con RDD activo por configuración global.

TABLA I
FASES DEL DESARROLLO Y ARTEFACTOS ODD

Fase	Rama	Fecha de merge	Commits	Diff (+/−)	Documento ODD
0	feature/concurrency-core	09/10/2026	7	+556 / −0	No
1	feature/ui-dashboard	09/10/2026	2	+133 / −5	No
2	feature/chaos-monkey-interrupts	11/10/2026	4	+506 / −14	Sí: 4 tareas, ruta en línea
3	feature/json-export	11/10/2026	3	+205 / −4	No (solo observación #28 en Engram)
4	feature/simulation-settings	11/10/2026	5	+1260 / −85	Sí: 4 tareas, escritor delegado

Nota: commits sin contar los merges; el diff es el del merge contra su primer padre. Fuente: historial de Git de la rama DesarrolloAPO (git log --first-parent, git diff --shortstat) y carpeta APO/odd/tasks.

C. Fuentes de evidencia e indicadores

Se usaron cinco fuentes: (1) los documentos ODD `chaos-monkey-interrupts.md` y `simulation-settings.md`; (2) el historial de Git (mensajes, hashes y estadísticas de cada commit); (3) las observaciones #26 a #32 de Engram en el proyecto `2026-progc-q2-m4` y las relaciones entre ellas; (4) la batería de pruebas, que se ejecutó una vez más para este informe; y (5) los registros de las revisiones RDD (identificador del linaje, lente, veredicto y hallazgos) que el agente copió en los documentos ODD. La Tabla II operacionaliza la hipótesis en cuatro indicadores y fija de antemano qué cuenta como evidencia a favor y en contra.

TABLA II
OPERACIONALIZACIÓN DE LOS INDICADORES

Indicador	Evidencia a favor de H	Evidencia en contra de H
Trazabilidad	Criterios de aceptación explícitos; cada tarea enlazada a pruebas (RED → GREEN) y a un hash de commit; la cadena se puede recorrer hacia atrás.	Fases sustanciales sin documento; tareas sin commit o sin prueba; criterios implícitos.
Revisión	Revisión independiente del candidato congelado, con hallazgos que las pruebas no detectaron y que luego se corrigieron.	Revisiones no realizadas, bloqueadas o sin efecto sobre el código.
Memoria	Una sesión retoma decisiones o descubrimientos guardados en otra; espejo del documento ODD en Engram; relaciones juzgadas entre observaciones.	Contexto reconstruido a mano; espejo ausente o incompleto; observaciones duplicadas o contradictorias.
Costo	Artefactos solo para trabajo sustancial; fricción acotada y resoluble.	Bloqueos del flujo, revisiones redundantes, archivos ajenos al producto, trabajo extra sin valor.

Nota: indicadores definidos antes del análisis del caso. Fuente: elaboración propia a partir de la hipótesis H.

D. Amenazas a la validez

- **Ausencia de grupo de control.** Sin una ejecución equivalente con SDD, ningún resultado del caso permite afirmar que ODD sea mejor o peor que SDD; solo si los mecanismos estuvieron presentes.
- **Caso único.** Un solo equipo, un solo proyecto y un único modelo limitan la generalización.
- **Registros escritos por el propio agente.** Los documentos ODD y las observaciones de Engram los redactó el agente evaluado. Para mitigarlo, se contrastaron con el historial de Git y con una nueva ejecución de las pruebas.
- **Deriva de versiones.** gentle-ai cambió de manera sustancial entre las versiones 3.0.0 y 4.2.0 en pocas semanas, por lo que los resultados describen la versión 4.0.0.
- **Sesgo del autor.** El grupo eligió la herramienta y evalúa su propio proceso.

- **Cumplimiento a nivel de prompt.** ODD no se aplica por código; su presencia depende de que el modelo siga las instrucciones, lo cual es a la vez una variable a observar y una fuente de variabilidad.

IV. RESULTADOS Y DISCUSIÓN

A. Comparación conceptual (PI1)

La Tabla III resume la comparación de ambos flujos en diez dimensiones, y la Fig. 1 contrasta sus estructuras. La diferencia de fondo está en qué se considera la fuente de verdad. En SDD es una base de especificaciones que se mantiene viva y que el código debe satisfacer. En ODD es el pedido del usuario más el código existente, y el documento de funcionalidad registra la intención y la evidencia de un cambio concreto, sin consolidarse en una base de especificaciones del sistema. En la clasificación de Böckeler [17], SDD en gentle-ai era *spec-anchored* y ODD, en el mejor de los casos, se acerca a *spec-first*; esta ubicación es una interpretación de los autores.

TABLA III
COMPARACIÓN CONCEPTUAL ENTRE SDD Y ODD EN GENTLE-AI

Dimensión	SDD (gentle-ai ≤ v3.7, Agent Teams Lite)	ODD (gentle-ai v4.0.0)
Unidad de planificación	El cambio, recorrido por nueve fases.	La tarea; el pedido se clasifica como pequeño o sustancial.
Entrada	Elección explícita o comandos /sdd-*.	Automática en cada pedido.
Artefactos	Propuesta, especificaciones delta, diseño, tareas e informe de verificación por cambio.	Ninguno para trabajo pequeño; un documento de funcionalidad y su espejo en Engram para el sustancial.
Momento de creación	Antes de escribir código, fase por fase, con aprobación entre fases.	Después de explorar y antes de la primera escritura, solo si el trabajo es sustancial.
Fuente de verdad	Base viva openspec/specs/, actualizada en archive (spec-anchored).	Pedido del usuario, código y documento de la funcionalidad; sin base de especificaciones.
Verificación	Fase verify contra la especificación; TDD estricto configurable.	Test-first por defecto con RED observado; verificaciones proporcionales por tarea.
Revisión	Integrada en el flujo (verify antes de archive).	Separada: RDD por commit o porción, según el riesgo, con consentimiento.
Persistencia	openspec/, Engram, híbrida o ninguna.	Archivo local más espejo en Engram, no atómicos.
Topología	Orquestador que solo delega; un sub-agente por fase.	En línea por defecto; delegación por disparadores (evidencia, escritura, contexto).
Costo de ceremonia	Alto y fijo: artefactos y traspasos en cada cambio.	Proporcional al tamaño; costo fijo del prompt de enrutamiento.
Modo de falla	Trabajo bloqueado por el estado del flujo (#4612).	Omisión del protocolo: depende de que el modelo siga el prompt.

Nota: elaboración propia a partir de la documentación de Agent Teams Lite y gentle-ai [20], [25], [24], [22].

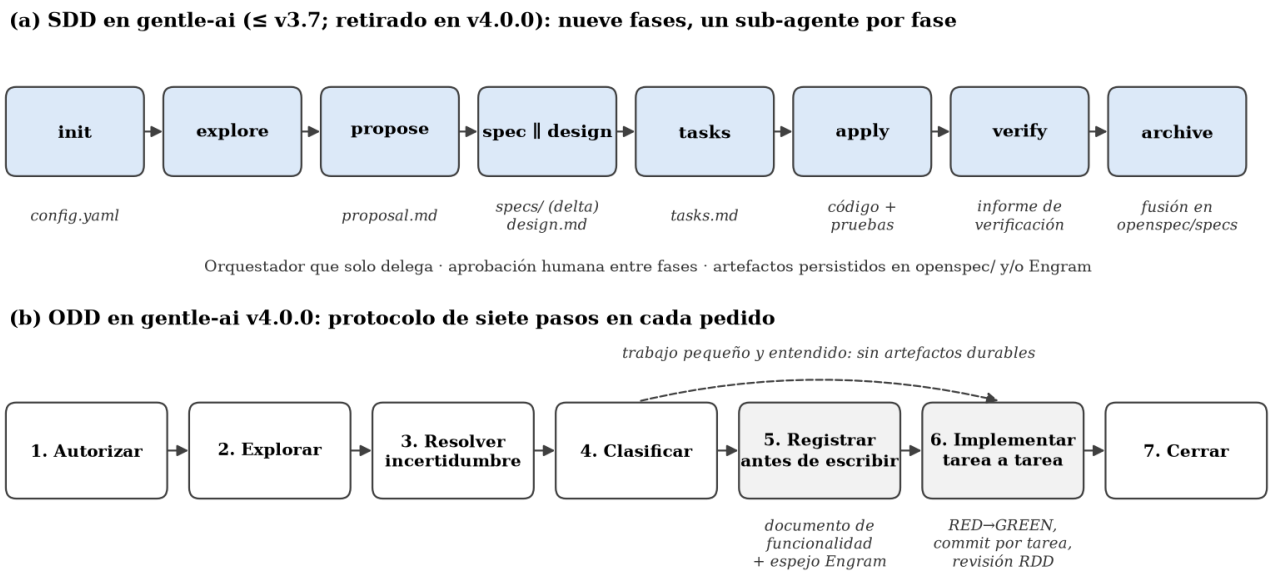


Fig. 1. Estructura de los dos flujos. (a) SDD en gentle-ai: fases secuenciales con un artefacto por fase. (b) ODD en gentle-ai v4.0.0: el paso de clasificación decide si se genera el documento de funcionalidad.

Ambos flujos comparten la exploración previa, las listas de tareas y la verificación antes de declarar un trabajo terminado. Difieren en tres aspectos. El primero es la proporcionalidad: SDD paga el mismo costo de artefactos para cualquier cambio que entre al flujo, mientras que ODD lo condiciona a la clasificación. El segundo es dónde residen las garantías: SDD las codifica en fases y transiciones del binario, y ODD en instrucciones del prompt más un mecanismo de revisión (RDD) que sí está implementado en código. El tercero es la continuidad: SDD la obtiene de los artefactos en disco y ODD la delega en el documento y en Engram. Los modos de falla son, por lo tanto, simétricos: SDD tiende al exceso de ceremonia y ODD a la omisión silenciosa.

B. Resultados del caso por indicador (PI2)

Trazabilidad. En las dos fases con documento ODD la cadena se puede recorrer completa. En la fase 4, el documento fija cuatro criterios de aceptación, entre ellos que una configuración o una opción inválida termine con código de salida 2 y un mensaje que nombre el campo. Ese criterio se vincula con las tareas T1 (validación) y T4a (endurecimiento de entradas), con las pruebas de `test_settings` y `test_cli`, y con los commits `c1e7151` y `f6e35a0`. La Tabla IV recoge la evidencia de cada tarea. Todas las tareas de las fases 2 y 4 registran un hash de commit, una ruta (en línea o delegada) y un resultado de pruebas observado; seis de las ocho registran además el estado RED previo a la implementación. Las dos restantes son excepciones previstas por el protocolo: la tarea T3 de la fase 2 (conexión del supervisor en `main.py`) se verificó con una ejecución manual de 8 s, y la tarea T4, que solo agrega una prueba de estrés, registró en su lugar un control negativo que la prueba detectó. En las fases 0, 1 y 3 no hay documento, criterios de aceptación ni registro RED: la trazabilidad se limita a los mensajes de commit y, en la fase 3, a la observación #28 de Engram. La fase 3 (tres commits, dos de funcionalidad y uno de pruebas) habría calificado como sustancial según la regla de la versión 4.0.0.

TABLA IV
EVIDENCIA TEST-FIRST REGISTRADA POR TAREA EN LAS FASES CON DOCUMENTO ODD

Tarea	Commit	Estado RED registrado	GREEN y suite completa
F2-T1	dba87ec	3 errores	Suite 12/12
F2-T2	35d7ee1	Error de importación	Suite 14/14
F2-T3	c8dd15c	— (ejecución manual de 8 s)	1 pausa, cierre por SIGINT, muelle vacío
F2-T4	890371f	— (tarea de solo pruebas; control negativo detectado)	Suite 17/17; 10 de 10 corridas
F4-T1	c1e7151	ModuleNotFoundError: settings	24/24 en test_settings; suite 46/46
F4-T2	aead10b	ModuleNotFoundError: cli	12/12 en test_cli; suite 58/58
F4-T3	4e6cc8c	5 errores y 1 falla en test_main_wiring	7/7; suite 65/65
F4-T4a	f6e35a0	7 fallas y 5 errores (nan/inf aceptados, OverflowError, UnicodeDecodeError)	Suite completa en verde
F4-T4b	9e9f690	Exportación con 37 paquetes frente a 38 finales	Suite 83/83, 4 ejecuciones

Nota: F2 = fase 2 (chaos-monkey-interrupts), F4 = fase 4 (simulation-settings). Fuente: secciones Evidence de APO/odd/tasks/*.md, contrastadas con git log.

Revisión independiente. En la fase 2, la evaluación de riesgo de RDD clasificó el cambio como de riesgo medio y con revisión debida (520 líneas), pero la revisión nunca se ejecutó. La verificación previa de gentle-ai 4.0.0 devolvió en todos los intentos el estado `intended_untracked_selection_required`, y cada envío de la selección solicitada fue rechazado (observación #27). El usuario decidió integrar la rama sin revisión, con 17/17 pruebas en verde; el código de esa fase solo quedó revisado después, dentro de las revisiones de rama completa de la fase 4. En la fase 4 el bloqueo se repitió y se resolvió listando los archivos no versionados de la carpeta `APO` (documentos de la cátedra, `AGENTS.md` y `odd/`) en el `.git/info/exclude` local (observación #31). A partir de allí se completaron cuatro revisiones, todas aprobadas y con su autorización quemada: una con la lente de fiabilidad sobre las tareas T1–T3 (889 líneas), una sobre la tarea T4 (510 líneas) y dos sobre la rama completa, de 2154 y 2552 líneas. Ninguna bloqueó la entrega, pero los hallazgos de carácter consultivo tuvieron consecuencias concretas. El usuario autorizó la tarea T4 para corregir cinco de ellos, que las pruebas existentes no detectaban:

- valores `nan`, `inf` o enteros enormes aceptados como segundos (`R3-nonfinite-seconds`);
- un archivo de configuración no decodificable como UTF-8 que terminaba en una traza de error en lugar de un mensaje legible (`R3-config-decode-traceback`);
- la exportación final del JSON realizada antes de que los operarios vaciaran el muelle (`R3-final-export-before-drain`);
- un cierre que no era seguro ante interrupciones (`R3-shutdown-not-interrupt-safe`);
- un estado de salida que no reflejaba la falla de un hilo trabajador (`R3-worker-failure-exit-status`).

Los tres últimos se originaron en fases anteriores —la exportación proviene de la fase 3— y los encontró la revisión de la rama completa, no la revisión del cambio. La corrección de la tarea T4b es la más ilustrativa: con el código anterior, el JSON exportado registraba 37 paquetes mientras que el resumen final informaba 38, una violación de orden entre el exportador y los consumidores del tipo que describen Lu et al. [7]. Se resolvió con un cierre en tres fases, en el que el exportador espera la señal `operators_drained` antes de la escritura final, que sigue siendo atómica [34]. Quedaron registradas como pendientes otras observaciones (código de salida 130 sin prueba, falla de `_on_stop` no contabilizada, enteros de más de 4300 dígitos en el JSON).

Memoria y continuidad. Las fases 2 a 4 dejaron siete observaciones en Engram (#26 a #32): tres decisiones, dos descubrimientos y dos resúmenes de sesión, distribuidos en tres sesiones. La evidencia de continuidad entre sesiones es directa: el bloqueo de revisión descubierto en la sesión de la fase 2 (#27) fue reconocido como «el mismo problema» en la sesión de la fase 4, y su solución (#31) se registró con una relación `supersedes` sobre la observación original. El agente juzgó en total nueve relaciones entre las observaciones del caso: siete `related`, una `compatible` y una `supersedes`. Los resúmenes de sesión (#30 y #32) dejaron explícitos los próximos pasos, entre ellos la integración pendiente de dos ramas apiladas, que se ejecutó después. El espejo del documento ODD, en cambio, no cumplió lo que pide el protocolo: la clave `odd/simulation-settings/tasks` (observación #29, cuatro revisiones) contiene un resumen del estado y los pendientes, no el documento completo, y la fase 2 guardó su decisión con la clave `concurrency/logistics-center/chaos-monkey`, sin espejo bajo `odd/`. Los documentos ODD, a su vez, quedaron excluidos del control de versiones como parte de la solución al bloqueo de revisión, de modo que su única copia duradera fuera de la máquina local es ese espejo parcial.

Estabilidad del resultado. La ejecución de la batería para este informe (`python3 -m unittest discover -s tests`) terminó con 83 pruebas correctas en 19,9 s. Sin embargo, tras la última integración se había observado una falla en la primera ejecución y 14 ejecuciones correctas consecutivas después (observación #29). Como el árbol integrado es idéntico al de la rama, se trata de una prueba dependiente de los tiempos; la sospechosa es el control negativo de `test_chaos`, que la revisión ya había señalado. La política test-first no eliminó ese tipo de fragilidad, característica de las pruebas de concurrencia.

C. Costos y fricciones (PI3)

TABLA V
COSTOS Y FRICCIONES OBSERVADOS EN EL CASO

Fricción	Evidencia	Efecto
Bloqueo de la revisión nativa	Preflight en intended_untracked_selection_required (obs. #27); resuelto con .git/info/exclude (obs. #31).	Fase 2 integrada sin revisión.
Archivos de configuración en el repositorio	La instalación de gentle-ai creó más de 120 archivos de agentes dentro del repositorio [1].	Exclusión manual del control de versiones.
Documentos ODD no versionados	APO/odd/ listado en .git/info/exclude.	Trazabilidad local; el espejo en Engram es parcial.
Cumplimiento no forzado	Fases 0, 1 y 3 sin documento ODD.	Sin criterios ni RED registrados.
Revisiones de rama completa	Un hook de cierre de sesión forzó dos revisiones de 2154 y 2552 líneas.	Mayor costo, pero detectaron 3 defectos previos.
Desvío de la estimación	Pronóstico de unas 450 líneas; T1–T3 sumaron 833 inserciones y 56 eliminaciones.	No consta la elección de estrategia de encadenamiento.
Prueba dependiente de los tiempos	1 falla y luego 14/14 ejecuciones correctas tras el merge (obs. #29).	Seguimiento pendiente.

Nota: fuentes indicadas en cada fila; documentos APO/odd/tasks/*.md y observaciones de Engram del proyecto 2026-progc-q2-m4.

El costo más visible no provino de ODD sino de la integración de RDD con un repositorio compartido que contiene archivos ajenos al código (enunciados, rúbricas y plantillas de la cátedra). La revisión de gentle-ai 4.0.0 exige seleccionar de forma exacta los archivos no versionados del candidato, y el agente no logró construir esa selección, por lo que hizo falta intervenir el entorno. La decisión de excluir los documentos ODD del control de versiones, tomada para destrabar la revisión, debilitó a su vez la trazabilidad que esos documentos aportan. Las revisiones de rama

completa duplicaron el volumen revisado, pero fueron las que encontraron los defectos más relevantes, por lo que su costo no puede considerarse puramente redundante. No se registró el consumo de tokens por fase en este ciclo, de modo que el costo computacional no se puede cuantificar con la evidencia disponible.

D. Discusión

Los resultados del caso coinciden con los argumentos de los autores de gentle-ai en ambos sentidos. A favor de ODD, el trabajo sustancial se documentó con un único archivo, sin propuestas, especificaciones delta ni diseños separados, y aun así conservó criterios de aceptación, RED → GREEN y hashes de commit. Eso coincide con el diagnóstico del issue #4612, según el cual la ceremonia de SDD agregaba puntos de bloqueo sin un valor proporcional [22]. También coincide con la economía de la delegación documentada en Agent Teams Lite, que estima un costo fijo de entre 11 850 y 12 866 tokens por cada sub-agente lanzado y un sobrecosto de alrededor del 1209 % al delegar una tarea de un solo archivo; los propios autores aclaran que son estimaciones a partir del tamaño de los archivos y no experimentos controlados [20]. En el caso, la fase 2 se resolvió en línea y la fase 4 con un escritor delegado, y ambas rutas quedaron registradas, que es justamente lo que el protocolo busca hacer observable.

En contra, el costo fijo de ODD se traslada al prompt. El issue #5090, abierto por un usuario sobre la versión 3.7.0, estima que el bloque de enrutamiento ocupa unos 18,2 KB (alrededor de 4500 tokens) que se vuelven a leer en cada turno, del orden de 2 millones de tokens de entrada en una sesión de 400 a 500 turnos [35]. Y la propia documentación de gentle-ai reconoce que sus pruebas de instrucciones «establecen la entrega, no el cumplimiento autónomo de cada paso de creación, actualización o reanudación» [25]. El caso lo confirma: tres de cinco fases no tuvieron documento y el espejo en Engram no reprodujo el documento completo. SDD, al imponer fases desde el binario, no habría permitido omitir los artefactos; habría trasladado el riesgo hacia el bloqueo del flujo, el modo de falla que motivó su retiro.

En cuanto a la memoria, el caso es coherente con la motivación de MemGPT y de los Generative Agents [32], [33]: el contexto relevante para la fase 4 (el bloqueo de revisión) no estaba en la ventana de la nueva sesión y se recuperó de un almacenamiento externo. Que la continuidad dependa de lo que el agente decide guardar es la contracara de la filosofía de curación de Engram: las observaciones fueron informativas y sin duplicados, pero el espejo omitió justamente el documento que debía preservar. Por último, la detección por revisión de defectos que las pruebas no cubrían —incluida una violación de orden en el cierre— es consistente con la dificultad de probar el código concurrente [7] y con la conclusión de la versión anterior de este informe [1]: la revisión independiente es el mecanismo de aseguramiento con mayor efecto observable en el caso, y ese mecanismo no forma parte de ODD sino de RDD.

V. CONCLUSIONES

La hipótesis resulta **parcialmente sustentada**. En las fases en que ODD se aplicó efectivamente (fases 2 y 4), el flujo conservó los cuatro mecanismos de aseguramiento que SDD obtenía con especificaciones formales: criterios de aceptación explícitos, seis de ocho tareas con RED registrado antes de la implementación (y excepciones justificadas en las otras dos), revisión independiente (ejecutada solo en la fase 4, aunque las revisiones de rama completa alcanzaron también el código de fases anteriores) y una cadena trazable entre criterio, tarea, prueba y commit. Lo hizo con un

único documento por funcionalidad, y Engram aportó una continuidad verificable entre sesiones. La hipótesis no se sostiene para las fases 0, 1 y 3, que no tuvieron documento ni registro test-first, y tampoco en lo que respecta al espejo en Engram, que no reprodujo el documento completo. Como no hubo grupo de control, estos resultados muestran que ODD *puede* proporcionar esos mecanismos, no que lo haga mejor o peor que SDD.

Para el grupo, el beneficio concreto de haber usado ODD fue doble: los documentos dejaron una evidencia de proceso auditable (rutas, RED → GREEN, revisiones y pendientes) que permitió escribir este análisis, y la combinación con RDD llevó a corregir cinco defectos de borde, tres de ellos heredados de fases anteriores. Los costos fueron la configuración del entorno, el bloqueo de la revisión y un cumplimiento irregular que dejó sin cobertura la mitad de las fases.

ODD resulta preferible cuando el trabajo es incremental, los cambios son en su mayoría pequeños o medianos y la continuidad entre sesiones importa más que una especificación duradera del sistema, como en este trabajo práctico. SDD, o una herramienta como Spec Kit, sería preferible cuando la especificación debe sobrevivir al cambio como contrato compartido, cuando hay varios equipos o cuando el riesgo de omitir pasos es mayor que el costo de la ceremonia.

Como trabajo futuro se proponen:

- una comparación controlada, con la misma consigna, el mismo modelo y la misma batería de pruebas, entre ODD y un flujo SDD externo (Spec Kit u OpenSpec);
- versionar los documentos ODD en el repositorio y verificar que el espejo en Engram contenga el documento completo;
- medir el consumo de tokens y el tiempo por fase con el mismo instrumento en todas las sesiones;
- estabilizar la prueba dependiente de los tiempos y repetir el análisis con gentle-ai 4.2.0, cuyas reglas de clasificación y delegación difieren de las aquí estudiadas.

VI. REFERENCIAS Y BIBLIOGRAFÍA

A. Referencias bibliográficas

[1] S. Alvarez, V. Blanco, S. Briasco Calvo et al., "Generación de código concurrente mediante VibeCoding: análisis crítico y métricas de un simulador de centro logístico en Python," Actividad Práctica Obligatoria N.° 1, Programación Concurrente, UNLaM, Versión 1.00, 15 oct. 2026.

[2] Gentleman Programming (A. Buscaglia). gentle-ai: The deterministic engineering environment for the AI agent you already use [en línea]. Consultado: 19 oct. 2026. Disponible: https://github.com/Gentleman-Programming/gentle-ai

[3] Gentleman Programming. engram: Persistent memory for AI coding agents [en línea]. README, DOCS.md y docs/ARCHITECTURE.md. Consultado: 19 oct. 2026. Disponible: https://github.com/Gentleman-Programming/engram

[4] Gentleman Programming. (15 oct. 2026). gentle-ai v4.2.0: Gentle Shell's routing ships in the provider contract [en línea]. Consultado: 19 oct. 2026. Disponible: https://github.com/Gentleman-Programming/gentle-ai/releases/tag/v4.2.0

[5] E. W. Dijkstra, "Cooperating sequential processes," en *Programming Languages*, F. Genuys, Ed. New York, NY, USA: Academic Press, 1968, pp. 43–112.
[6] C. A. R. Hoare, "Monitors: An operating system structuring concept," *Commun. ACM*, vol. 17, no. 10, pp. 549–557, oct. 1974.
[7] S. Lu, S. Park, E. Seo e Y. Zhou, "Learning from mistakes: A comprehensive study on real world concurrency bug characteristics," en *Proc. 13th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Seattle, WA, USA, 2008, pp. 329–339.
[8] M. Herlihy y N. Shavit, *The Art of Multiprocessor Programming*. Burlington, MA, USA: Morgan Kaufmann (Elsevier), 2008.
[9] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li et al., "Large language models for software engineering: A systematic literature review," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 8, pp. 1–79, nov. 2024.
[10] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt y R. Karri, "Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions," en *Proc. IEEE Symp. Security and Privacy (SP)*, San Francisco, CA, USA, 2022, pp. 754–768.
[11] D. Nichols, J. H. Davis, Z. Xie, A. Rajaram y A. Bhatele, "Can large language models write parallel code?," en *Proc. 33rd Int. Symp. High-Performance Parallel and Distributed Computing (HPDC)*, Pisa, Italia, 2024, pp. 281–294.
[12] A. Karpathy. (2 feb. 2025). There's a new kind of coding I call "vibe coding" [en línea]. Consultado: 19 oct. 2026. Disponible: https://x.com/karpathy/status/1886192184808149383
[13] R. Sapkota, K. I. Roumeliotis y M. Karkee, "Vibe coding vs. agentic coding: Fundamentals and practical implications of agentic AI," arXiv:2505.19443, may. 2025.
[14] S. Yao, J. Zhao, D. Yu, N. Du et al., "ReAct: Synergizing reasoning and acting in language models," arXiv:2210.03629, oct. 2022.
[15] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan y O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," arXiv:2405.15793, may. 2024.
[16] C. E. Jimenez, J. Yang, A. Wettig, S. Yao et al., "SWE-bench: Can language models resolve real-world GitHub issues?," arXiv:2310.06770, oct. 2023.
[17] B. Böckeler. (15 oct. 2025). Understanding spec-driven-development: Kiro, spec-kit, and Tessl [en línea]. martinfowler.com. Consultado: 19 oct. 2026. Disponible: https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html
[18] D. Delimarsky. (2 sep. 2025). Spec-driven development with AI: Get started with a new open source toolkit [en línea]. The GitHub Blog. Consultado: 19 oct. 2026. Disponible: https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/
[19] Kiro. Specs, Kiro Documentation [en línea]. Consultado: 19 oct. 2026. Disponible: https://kiro.dev/docs/specs/
[20] Gentleman Programming. agent-teams-lite (archivado): docs/architecture.md, docs/persistence.md, docs/concepts.md y docs/token-economics.md [en línea]. Consultado: 19 oct. 2026. Disponible: https://github.com/Gentleman-Programming/agent-teams-lite
[21] Gentleman Programming. (16 sep. 2026). gentle-ai v3.0.0: Organic Driven Development as the default workflow [en línea]. Consultado: 19 oct. 2026. Disponible: https://github.com/Gentleman-Programming/gentle-ai/releases/tag/v3.0.0
[22] A. Buscaglia. (15 sep. 2026). refactor(sdd): remove mandatory runtime governance and keep verification optional, issue #4612, Gentleman-Programming/gentle-ai [en línea]. Consultado: 19 oct. 2026. Disponible: https://github.com/Gentleman-Programming/gentle-ai/issues/4612
[23] A. Buscaglia. (24 sep. 2026). refactor(workflow): retire SDD/OpenSpec as a live workflow, issue #4959, Gentleman-Programming/gentle-ai [en línea]. Consultado: 19 oct. 2026. Disponible: https://github.com/Gentleman-Programming/gentle-ai/issues/4959

[24] Gentleman Programming. (1 oct. 2026). gentle-ai v4.0.0: SDD retires, ODD leads, module path moves to /v4 [en línea]. Consultado: 19 oct. 2026. Disponible: https://github.com/Gentleman-Programming/gentle-ai/releases/tag/v4.0.0
[25] Gentleman Programming. gentle-ai: docs/usage.md, docs/intended-usage.md y docs/trigger-rules.md (commit 4acaf42) [en línea]. Consultado: 19 oct. 2026. Disponible: https://github.com/Gentleman-Programming/gentle-ai/tree/main/docs
[26] A. Buscaglia. (15 sep. 2026). feat(odd): make ODD the orchestrator's mandatory default protocol in every runtime, issue #4643, Gentleman-Programming/gentle-ai [en línea]. Consultado: 19 oct. 2026. Disponible: https://github.com/Gentleman-Programming/gentle-ai/issues/4643
[27] A. Buscaglia. (14 sep. 2026). feat(odd): add automatic feature tasks, durable resume, and selective assumption checks, issue #4607, Gentleman-Programming/gentle-ai [en línea]. Consultado: 19 oct. 2026. Disponible: https://github.com/Gentleman-Programming/gentle-ai/issues/4607
[28] Gentleman Programming. (17 sep. 2026). gentle-ai v3.1.0: ODD closes each task with a work-unit commit and reviews per commit or slice [en línea]. Consultado: 19 oct. 2026. Disponible: https://github.com/Gentleman-Programming/gentle-ai/releases/tag/v3.1.0
[29] Gentleman Programming. gentle-ai: docs/architecture/organic-rdd.md y docs/review-integration.md (commit 4acaf42) [en línea]. Consultado: 19 oct. 2026. Disponible: https://github.com/Gentleman-Programming/gentle-ai/tree/main/docs
[30] Gentleman Programming. (22 sep. 2026). gentle-ai v3.5.0: Receipt-Driven Development on by default [en línea]. Consultado: 19 oct. 2026. Disponible: https://github.com/Gentleman-Programming/gentle-ai/releases/tag/v3.5.0
[31] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni y P. Liang, "Lost in the middle: How language models use long contexts," *Trans. Assoc. Comput. Linguistics*, vol. 12, pp. 157–173, 2024, doi: 10.1162/tacl_a_00638.
[32] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica y J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," arXiv:2310.08560, oct. 2023.
[33] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris et al., "Generative agents: Interactive simulacra of human behavior," en *Proc. 36th Annu. ACM Symp. User Interface Software and Technology (UIST '23)*, 2023, pp. 1–22, doi: 10.1145/3586183.3606763.
[34] T. S. Pillai, V. Chidambaram, R. Alagappan, S. Al-Kiswany, A. C. Arpaci-Dusseau y R. H. Arpaci-Dusseau, "All file systems are not created equal: On the complexity of crafting crash-consistent applications," en *Proc. 11th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, Broomfield, CO, USA, 2014, pp. 433–448.
[35] juliangarciaclaro. (29 sep. 2026). perf(routing): deduplicate the agent-routing CLAUDE.md block — ~4.5k tokens re-read every turn, issue #5090, Gentleman-Programming/gentle-ai [en línea]. Consultado: 19 oct. 2026. Disponible: https://github.com/Gentleman-Programming/gentle-ai/issues/5090

B. Bibliografía

G. R. Andrews, *Foundations of Multithreaded, Parallel, and Distributed Programming*. Reading, MA, USA: Addison-Wesley, 2000.
GitHub. spec-kit [en línea]. Consultado: 19 oct. 2026. Disponible: https://github.com/github/spec-kit
M. Chen, J. Tworek, H. Jun et al., "Evaluating large language models trained on code," arXiv:2107.03374, jul. 2021.